Databases

Matt ter Steege
matttersteege@gmail.com
Universiteit Utrecht
Utrecht, Nederland

Contents

Contents 1
1 Relationele model & algebra 2
1.1 defenities 2
1.2 Keys 2
1.3 Selectie & Projectie 2
2 Enity Relationship diagrams (ERD) 3
3 Functional dependencies & normalisatie 3
3.1 Normalisatietheorie 3
3.2 Functional dependencies 3
3.3 Decompositie 4
4 SQL 5
5 Normalisatie 6
5.1 Indexen 6
5.2 Boyce-Codd Normal Form (BCNF) 6
5.3 Minimal cover calculation 8
5.4 3NF 9
5.5 4NF 9

1 Relationele model & algebra

1.1 defenities

Relatieschema: Movie			
movid	**title**	**year**	**rating**
tt0469494	There will be blood	2007	8,1
tt0086879	Amadeus	1984	8,4
tt0102926	The silence of the lambs	1991	8,6
...	...	...	...

Uitleg gekoppeld aan de tabel:

- **Movie** is het **relatieschema** R R(movid, title, year, rating)
- De **kolomnamen** zijn de **attributen** $attr(R) = \{\text{movid, title, year, rating}\}$
- Het aantal attributen is 4, dus de **degree** is 4
- Elke rij in de tabel is één **tuple** Bijvoorbeeld: < tt0469494, There will be blood, 2007, 8,1 >
- Elk attribuut heeft zijn **domein**:
  - movid → string
  - title → string
  - year → integer
  - rating → float
- Alle rijen samen vormen de **relatie** $r(Movie)$

1.2 Keys

- **Key** is een attribuut of combinatie van attributen die een tuple uniek identificeert binnen een relatie. Bijvoorbeeld: movid is een key voor Movie.
- **Superkey** is een attribuut of combinatie van attributen die een tuple uniek identificeert, maar kan ook extra attributen bevatten. Bijvoorbeeld: (movid, title) is een superkey voor Movie.
  Definitie: Een set van attributen $K \subseteq attr(R)$ is een superkey als voor elke relatie $r(R)$: $\forall t_1, t_2 \in r(R) : t_1[K] = t_2[K] \Rightarrow t_1 = t_2$. of: "Als twee tuples dezelfde waarden hebben voor alle attributen in K, dan zijn het dezelfde tuples."
- **Candidate key** is een minimale superkey, wat betekent dat het geen extra attributen bevat die niet nodig zijn voor unieke identificatie. Bijvoorbeeld: movid is een candidate key voor Movie. Definitie: Een set van attributen $K \subseteq attr(R)$ is een candidate key als K een superkey is en geen echte subset van K een superkey is. of: "Er is geen kleinere subset van attributen die ook een superkey is."
- **Primary key** is de gekozen candidate key die wordt gebruikt om tuples uniek te identificeren binnen een relatie. Bijvoorbeeld: movid is de primary key voor Movie.
- **Foreign key** is een attribuut of combinatie van attributen in een relatie die verwijst naar de primary key van een andere relatie. Bijvoorbeeld: in een relatie Actor met attribuut movid als foreign key die verwijst naar de primary key movid in Movie. Dit kan je gebruiken om aan te geven in welke films een acteur heeft gespeeld. Definitie: Een set attributen $K \in attr(R)$ is een foreign key van relatie R_j als candidate key. $R_i[K]$ verwijst naar de primary key van relatie R_j. of: "De waarden van K in R_i moeten overeenkomen met de waarden van de primary key in R_j."
- **Key constraints** zorgen ervoor dat de waarden van de key-attributen uniek zijn binnen de relatie en niet null mogen zijn.
- De **referential integrity constraints** zorgen ervoor dat de waarden van de foreign key-attributen overeenkomen met de waarden van de primary key in de gerelateerde relatie, of null zijn als er geen overeenkomst is.

1.3 Selectie & Projectie

- **Selectie** (σ_p), σ is een operator die rijen (tuples) selecteert uit een relatie op basis van het predicaat p.
- **and** ($\wedge$) en **or** ($\vee$) worden gebruikt om meerdere voorwaarden te combineren in het predicaat. Voorbeeld: $\sigma_{\text{year}>2000 \wedge \text{rating}>8}(Movie)$ selecteert alle films uit de Movie-relatie die zijn uitgebracht na het jaar 2000 en een beoordeling hoger dan 8 hebben.
- **Projectie** (π_L), π is een operator die kolommen (attributen) selecteert uit een relatie op basis van de lijst van attributen L. Voorbeeld: $\pi_{\text{title,year}}(Movie)$ projecteert alleen de titel en het jaar van de films uit de Movie-relatie.
- **union** ($\cup$), de unie-operator combineert de resultaten van twee relaties door alle unieke tuples uit beide relaties te behouden. Voorbeeld: $R_1 \cup R_2$ geeft alle unieke tuples die voorkomen in zowel relatie R_1 als relatie R_2.
- **intersection** ($\cap$), de intersectie-operator retourneert de tuples die voorkomen in beide relaties. Voorbeeld: $R_1 \cap R_2$ geeft alle tuples die in zowel relatie R_1 als relatie R_2 voorkomen.
- **set difference** ($-$), de verschil-operator retourneert de tuples die voorkomen in de eerste relatie maar niet in de tweede. Voorbeeld: $R_1 - R_2$ geeft alle tuples die in relatie R_1 voorkomen maar niet in relatie R_2.

- **cartesian product** ($\times$), het cartesisch product combineert elke tuple van de eerste relatie met elke tuple van de tweede relatie. Voorbeeld: $R_1 \times R_2$ geeft alle mogelijke combinaties van tuples uit relatie R_1 en relatie R_2.
- **thetha-join** ($\bowtie_\theta$), de theta-join-operator combineert tuples uit twee relaties op basis van een opgegeven voorwaarde θ. Voorbeeld: $R_1 \bowtie_{\text{R1.attr=R2.attr}} R_2$ combineert tuples uit relatie R_1 en relatie R_2 waar de waarden van het attribuut attr gelijk zijn.
- **natural join** ($\bowtie$), de natural join-operator combineert tuples uit twee relaties op basis van gemeenschappelijke attributen met dezelfde naam. Voorbeeld: $R_1 \bowtie R_2$ combineert tuples uit relatie R_1 en relatie R_2 waar de waarden van de gemeenschappelijke attributen gelijk zijn.
- **division** ($\div$), de division-operator retourneert de tuples uit de eerste relatie die overeenkomen met alle tuples in de tweede relatie. Voorbeeld: $R_1 \div R_2$ geeft alle tuples uit relatie R_1 die overeenkomen met alle tuples in relatie R_2 op basis van een gemeenschappelijk attribuut.
- **Assignment & renaming** $T :=<\text{alg}_e xpr >, deassignment-operatorwijsthetresultaatvaneenrelationelealgebra-expressietoeaaneennieuwerelatie\text{T.V}$ $:= \sigma_{\text{year}>2000}(Movie)$ creëert een nieuwe relatie T die alle films bevat die zijn uitgebracht na het jaar 2000.
- **Renaming on the fly** $(\rho(T)(<\text{alg}_e xpr >), derenaming-operatorwijsthetresultaatvaneenrelationelealgebra-expressietoeaaneennieuwerelatie\text{Tzond}$ creëert een tijdelijke relatie T die alle films bevat die zijn uitgebracht na het jaar 2000, maar deze relatie is alleen beschikbaar binnen de context van deze expressie.

Voorbeeld **(bibliotheek schema)**:

Stel je hebt de volgende schema's voor een bibliotheekdatabase:

- **Book**(book_id, title, author, year)
- **Member**(member_id, name, join_date)
- **Loan**(loan_id, book_id, member_id, loan_date, return_date)

Geef alle namen van lezers die minstens één boek hebben geleend van Dickens.

$$\pi_{\text{name}}(\text{Reader} \bowtie (\text{Loan} \bowtie \sigma_{\text{author}='Dickens'}(\text{Book})))$$

Uitleg:

- $\sigma_{\text{author}='Dickens'}(\text{Book})$ selecteert alle boeken geschreven door Dickens.
- $\text{Loan} \bowtie \sigma_{\text{author}='Dickens'}(\text{Book})$ combineert de leningen met de geselecteerde boeken om alle leningen van Dickens' boeken te vinden.
- $\text{Reader} \bowtie (\text{Loan} \bowtie \sigma_{\text{author}='Dickens'}(\text{Book}))$ combineert de lezers met de leningen van Dickens' boeken om alle lezers te vinden die deze boeken hebben geleend.
- $\pi_{\text{name}}(\ldots)$ projecteert alleen de namen van deze lezers.

---

2 Enity Relationship diagrams (ERD)

Als je een database wilt maken, dan is het belangrijk dat je het domein goed modelleert. Een ERD is een grafische weergave van de entiteiten, attributen en relaties binnen een domein. Het helpt bij het ontwerpen van databases door de structuur en de onderlinge verbanden duidelijk te maken.

ER diagrammen hebben verschillende componenten:

- **Rechthoek** Geeft een entiteit weer, bijvoorbeeld een persoon, plaats of ding.
- **Ovaal** Geeft een attribuut weer, bijvoorbeeld een naam, leeftijd of adres.
- **Ruit** Geeft een relatie weer tussen entiteiten, bijvoorbeeld "werkt bij" of "heeft".
- **Lijn** Verbindt entiteiten, attributen en relaties om de structuur van het domein te tonen.

Deze lijnen/relaties kunnen ook kardinaliteiten hebben, zoals 1:1, 1:N of M:N, om aan te geven hoeveel entiteiten er betrokken zijn bij de relatie.
1:1 (één-op-één) betekent dat één entiteit van het ene type overeenkomt met precies één entiteit van het andere type.
1:N (één-op-veel) betekent dat één entiteit van het ene type overeenkomt met meerdere entiteiten van het andere type.
M:N (veel-op-veel) betekent dat meerdere entiteiten van het ene type overeenkomen met meerdere entiteiten van het andere type.

Ook kan je deelname aan een relatie aangeven als verplicht (1,...) of optioneel (0,...), wat aangeeft of een entiteit deel moet nemen aan de relatie of niet.

Zwakke entiteiten kunnen alleen bestaan in relatie tot een andere entiteit, en hebben een gedeeltelijke sleutel die samen met de sleutel van de gerelateerde entiteit een unieke identificatie vormt. Bijvoorbeeld een afboeking als je iets koopt kan alleen als je een bankrekening hebt, en de afboeking heeft een gedeeltelijke sleutel die samen met de rekeningnummer van de bankrekening een unieke identificatie vormt.

3 Functional dependencies & normalisatie

3.1 Normalisatietheorie

Wanneer is een databaseontwerp goed?

3.2 Functional dependencies

Stel je hebt een universum $R(A, B, C, D)$. Je hebt de regels $A \to B, B \to C, A \to C$

Dan kan je concluderen dat $A \to AB \to ABC$, ofwel $A \to ABC$.

Stel je hebt $A \to BCD$, dan kan je zeggen $A \to B$

Voorbeeld **(Voorbeeld van functionele afhankelijkheden)**:

We hebben een schema $R(ABCDEFG)$ en een set FD's: $F = C \to DE, A \to C, G \to D, B \to G$
We willen de closure van A berekenen, oftewel A^+, om te zien welke attributen functioneel afhankelijk zijn van A.
We beginnen met $A^+ = \{A\}$ (omdat A altijd functioneel afhankelijk is van zichzelf).
We kijken naar de FD's in F en zien dat $A \to C$ in F zit, dus we voegen C toe aan A^+: $A^+ = \{A, C\}$.
Nu kijken we opnieuw naar de FD's en zien dat $C \to DE$ in F zit, en aangezien C al in A^+ zit, kunnen we D en E ook toevoegen aan A^+: $A^+ = \{A, C, D, E\}$.
We kijken nogmaals naar de FD's, maar er zijn geen FD's meer die we kunnen toepassen op de attributen in A^+, dus we zijn klaar.
Stel wel willen $(AB)^+$ weten (notitie, dat is dus NIET AB^+, want dat is $A(B)^+$)

3.3 Decompositie

Voorbeeld **(XYZ-regel (Verliesvrij))**:

Gegeven $R(ABCDE)$ en FD's $F = \{A \to C,\ C \to BE,\ A \to E\}$.
Beschouw de decompositie $R_1 = (AC)$ en $R_2 = (ABDE)$.

De gemeenschappelijke attributen zijn $X = R_1 \cap R_2 = \{A\}$.
Schrijf $Y = R_1 \setminus X = \{C\}$ en $Z = R_2 \setminus X = \{B, D, E\}$.

Volgens de XYZ-regel is de decompositie verliesvrij als $X \to Y$ of $X \to Z$ geldt (met betrekking tot F^+).
Controleer $A \to C$ en $A \to BDE$:
uit F volgt $A \to C$ en $A \to E$, maar er is geen $A \to B$ en geen $A \to D$.
Dus geldt $A \to C$ maar niet $A \to BDE$.

Conclusie: de decompositie (AC) $(ABDE)$ is verliesvrij volgens de gegeven FD's.

Voorbeeld **(XYZ-regel (niet verliesvrij))**:

Gegeven $R(ABCDE)$ en FD's $F = \{A \to C,\ C \to BE,\ A \to E\}$.
Beschouw de decompositie $R_1 = (AC)$ en $R_2 = (CBDE)$.

De gemeenschappelijke attributen zijn $X = R_1 \cap R_2 = \{C\}$.
Schrijf $Y = R_1 \setminus X = \{A\}$ en $Z = R_2 \setminus X = \{B, D, E\}$.

Volgens de XYZ-regel is de decompositie verliesvrij als $X \to Y$ of $X \to Z$ geldt (met betrekking tot F^+).
Controleer $C \to A$ en $C \to BDE$:
uit F volgt $C \to B$ en $C \to E$, maar er is geen $C \to A$ en geen $C \to D$.
Dus geldt noch $C \to A$ noch $C \to BDE$.

Conclusie: de decompositie (AC) $(CBDE)$ is niet verliesvrij volgens de gegeven FD's.

4 SQL

SQL (Structured Query Language) is een gestandaardiseerde programmeertaal die wordt gebruikt voor het beheren en manipuleren van relationele databases. SQL biedt een krachtige en flexibele manier om gegevens op te vragen, in te voegen, bij te werken en te verwijderen, evenals om de structuur van databases te definiëren en te beheren.

Om SQL uit te leggen gebruik ik deze tabel als voorbeeld:

CREATE TABLE wordt gebruikt om een nieuwe tabel in de database te maken.

```sql
CREATE TABLE Books ( -- de naam van de tabel
    book_id INT PRIMARY KEY, -- een integer kolom die dient als primaire sleutel
    title VARCHAR(255) NOT NULL, -- string, maximaal 255 tekens, mag niet null zijn
    author VARCHAR(255) NOT NULL,
    year INT,
    genre VARCHAR(100)
);
```

↓

book_id	title	author	year	genre

INSERT INTO wordt gebruikt om nieuwe rijen (records) toe te voegen aan een tabel. Je moet strings tussen aanhalingstekens zetten, en getallen niet.

```sql
INSERT INTO Books (book_id, title, author, year, genre) VALUES -- naam en kolommen waarin je wilt invoegen
(1, 'The Great Gatsby', 'F. Scott Fitzgerald', 1925, 'Fiction'),
(2, 'To Kill a Mockingbird', 'Harper Lee', 1960, 'Fiction'),
(3, 'A Brief History of Time', 'Stephen Hawking', 1988, 'Science'),
(4, 'The Art of Computer Programming', 'Donald Knuth', 1968, 'Computer Science'),
(5, 'The Lord of the Rings', 'J.R.R. Tolkien', 1954, 'Fantasy');
```

↓

book_id	title	author	year	genre
1	The Great Gatsby	F. Scott Fitzgerald	1925	Fiction
2	To Kill a Mockingbird	Harper Lee	1960	Fiction
3	A Brief History of Time	Stephen Hawking	1988	Science
4	The Art of Computer Programming	Donald Knuth	1968	Computer Science
5	The Lord of the Rings	J.R.R. Tolkien	1954	Fantasy

UPDATE wordt gebruikt om bestaande records in een tabel bij te werken. Je moet een WHERE-clausule gebruiken om aan te geven welke records je wilt bijwerken, anders worden alle records bijgewerkt.

```sql
UPDATE Books
SET genre = 'Classic Fiction' -- de nieuwe waarde die je wilt instellen
WHERE book_id = 1; -- de voorwaarde om te bepalen welke record(s) worden bijgewerkt
```

↓

book_id	title	author	year	genre
1	The Great Gatsby	F. Scott Fitzgerald	1925	**Classic Fiction**
2	To Kill a Mockingbird	Harper Lee	1960	Fiction
3	A Brief History of Time	Stephen Hawking	1988	Science
4	The Art of Computer Programming	Donald Knuth	1968	Computer Science
5	The Lord of the Rings	J.R.R. Tolkien	1954	Fantasy

DELETE wordt gebruikt om records uit een tabel te verwijderen. Je moet een WHERE-clausule gebruiken om aan te geven welke records je wilt verwijderen, anders worden alle records verwijderd.

```
DELETE FROM Books
WHERE book_id = 2; -- de voorwaarde om te bepalen welke record(s) worden verwijderd
```

↓

book_id	title	author	year	genre
1	The Great Gatsby	F. Scott Fitzgerald	1925	Classic Fiction
3	A Brief History of Time	Stephen Hawking	1988	Science
4	The Art of Computer Programming	Donald Knuth	1968	Computer Science
5	The Lord of the Rings	J.R.R. Tolkien	1954	Fantasy

SELECT wordt gebruikt om gegevens op te vragen uit een tabel. Je kunt specifieke kolommen selecteren, alle kolommen selecteren, en je kunt ook voorwaarden gebruiken om te filteren welke rijen worden geretourneerd.

```
SELECT title, author
FROM Books
WHERE genre = 'Science'; -- de voorwaarde om te bepalen welke record(s) worden geselecteerd
```

↓

title	author
A Brief History of Time	Stephen Hawking

Andere keywords die je vaak zult tegenkomen in SQL zijn:

- **JOIN** wordt gebruikt om rijen van twee of meer tabellen te combineren op basis van een gerelateerde kolom tussen hen.
- **GROUP BY** wordt gebruikt in combinatie met aggregatiefuncties (zoals COUNT, SUM, AVG, MAX, MIN) om rijen te groeperen die dezelfde waarden hebben in opgegeven kolommen.
- **ORDER BY** wordt gebruikt om de resultaten van een query te sorteren op basis van een of meer kolommen, in oplopende (ASC) of
- **IN** wordt gebruikt in een WHERE-clausule om te controleren of een waarde overeenkomt met een waarde in een lijst van waarden.
- **LIKE** wordt gebruikt in een WHERE-clausule om te zoeken naar een opgegeven patroon in een kolom.
- **DISTINCT** wordt gebruikt in een SELECT-query om dubbele rijen uit het resultaat te verwijderen, zodat alleen unieke
- **EXISTS** wordt gebruikt in een WHERE-clausule om te controleren of een subquery resultaten oplevert.

5 Normalisatie

Tijdens het normaliseren is het de bedoeling dat je de database opsplitst in meerdere tabellen om: **1.** redundantie te verminderen, **2.** lossless te zijn.

5.1 Indexen

Indexen zijn datastructuren die worden gebruikt om de snelheid van gegevensopvragingen in een database te verbeteren. Ze werken als een soort inhoudsopgave voor de database, waardoor het sneller is om specifieke records te vinden zonder de hele tabel te hoeven doorzoeken. Indexen kunnen worden gemaakt op één of meer kolommen van een tabel, en ze kunnen verschillende vormen aannemen, zoals B-tree indexen, hash indexen, of bitmap indexen, afhankelijk van het type gegevens en de query's die worden uitgevoerd. Het gebruik van indexen kan de prestaties van zoekopdrachten aanzienlijk verbeteren, vooral bij grote datasets, maar het kan ook extra opslagruimte vereisen en de prestaties van schrijfoperaties (zoals INSERT, UPDATE, DELETE) verminderen, omdat de indexen ook moeten worden bijgewerkt.

5.2 Boyce-Codd Normal Form (BCNF)

*Als iets in BCNF is, dan is het ook in 3NF, maar als iets in 3NF is, dan is het niet per se in BCNF.

De **Boyce-Codd Normal Form (BCNF)** is een strengere vorm van normalisatie dan de derde normaalvorm (3NF). Een relatie is in BCNF als voor elke functionele afhankelijkheid $X \rightarrow Y$ geldt dat X een superkey is. Dit betekent dat er geen functionele afhankelijkheden mogen zijn waarbij de determinant X geen superkey is. BCNF elimineert dus alle anomalieën die kunnen ontstaan door functionele afhankelijkheden waarbij de determinant geen superkey is, waardoor het ontwerp van de database robuuster en efficiënter wordt.

Voorbeeld **(BCNF voorbeeld)**:

Student	Teacher	Course
Alice	Dr. Smith	Math
Alice	Dr. Jones	History
Bob	Dr. Smith	Math
Bob	Dr. Stoned	History

Hieruit kunnen we opmaken dat de FD's zijn:

- (Student, Course) → Teacher (omdat elke student in een bepaald vak maar één docent heeft)
- (Student, Teacher) → Course (omdat elke student bij een bepaalde docent maar één vak volgt)
- Teacher → Course (omdat elke docent maar één vak geeft)

Hierdoor zijn de candidate keys (Student, Course) en (Student, Teacher), maar Teacher is geen superkey omdat er meerdere docenten zijn die hetzelfde vak geven. Dus deze relatie is niet in BCNF. We kunnen deze relatie decomponeren in twee relaties:

- R1(Teacher, Course) met de FD Teacher → Course
- R2(Student, Teacher) met de FD (Student, Teacher) → Course

Nu zijn beide relaties in BCNF, omdat in R1 Teacher een superkey is en in R2 (Student, Teacher) een superkey is.

5.3 Minimal cover calculation

Een minimal cover van een set functionele afhankelijkheden is een equivalentie van die set waarbij alle functionele afhankelijkheden in de minimal cover minimaal zijn, wat betekent dat er geen overbodige functionele afhankelijkheden zijn en dat de rechterkant van elke functionele afhankelijkheid slechts één attribuut bevat. Het berekenen van een minimal cover gaat als volgt:
Stel je hebt de volgende set functionele afhankelijkheden:

$$F = \{A \rightarrow D, B \rightarrow HA, GH \rightarrow AKL, CH \rightarrow BK, G \rightarrow BC\}$$

Calculate a minimal cover for **F**

Stap 1 is om de rechterkant van elke functionele afhankelijkheid te splitsen in enkelvoudige attributen:

- A → D
- B → H
- B → A
- GH → A
- GH → K
- GH → L
- CH → B
- CH → K
- G → B
- G → C

Stap 2 is de linkerkant van elke functionele afhankelijkheid minimaliseren door te controleren of er overbodige attributen zijn:

- A → D
- B → H
- B → A
- GH → A ($G^+ = GBCHAKLD$, dus G → A)
- GK → A ($G^+ = GBCHAKLD$, dus G → K)
- GL → A ($G^+ = GBCHAKLD$, dus G → L)
- CH → B (Kan niet versimpeld worden, want $C^+ \rightarrow C$ en $H^+ \rightarrow H$)
- CH → K (Kan niet versimpeld worden, want $C^+ \rightarrow C$ en $H^+ \rightarrow H$)
- G → B
- G → C

Stap 3 is om te kijken over er overbodige functionele afhankelijkheden zijn door te controleren of ze kunnen worden afgeleid uit de andere functionele afhankelijkheden:

- A → D
- B → H
- B → A
- G → A (kan worden afgeleid uit G → B en B → A, dus overbodig)
- G → K (kan worden afgeleid uit G → C en C → K, dus overbodig)
- G → L (kan *niet meer* worden afgeleid, want G → A is al weg)
- CH → B
- CH → K
- G → B
- G → C

De minimal cover van F is dus:

$$F_{min} = \{A \rightarrow D, B \rightarrow HA, G \rightarrow LBC, CH \rightarrow BK\}$$

(opmerking: G → LBC is een notatie die betekent dat G → L, G → B en G → C allemaal in de minimal cover zitten)

5.4 3NF

5.5 4NF

*Als iets in 4NF is, dan is het ook in BCNF, maar als iets in BCNF is, dan is het niet per se in 4NF!

De vierde normaalvorm probeert multivalued dependencies te elimineren, wat een situatie is waarin een attribuut afhankelijk is van een andere attribuut, maar niet van de hele sleutel. Een relatie is in 4NF als voor elke multivalued dependency $X \twoheadrightarrow Y$ geldt dat X een superkey is. Dit betekent dat er geen multivalued dependencies mogen zijn waarbij de determinant X geen superkey is, waardoor het ontwerp van de database nog robuuster wordt en verdere anomalieën worden voorkomen.

Voorbeeld **(4NF voorbeeld)**:

Persoon	Huisdier	Auto
Alice	Hond	BMW
Alice	Kat	BMW
Alice	Hond	Audi
Alice	Kat	Audi
Bob	Hond	Audi
Bob	Kat	Audi

Als je dit leest, dan kan je denken dat Alice heel rijk is, 4 huisdieren en 4 dure auto's. Maar niks is minder waar, want Alice heeft *maar* 2 huisdieren en 2 auto's. Namelijk, Alice heeft een hond en een kat, en ze heeft een BMW en een Audi. Maar vanwege het cartesisch product van de huisdieren en auto's in de tabel, lijkt het alsof Alice 4 huisdieren en 4 auto's heeft, terwijl ze er maar 2 van elk heeft. Dit is een voorbeeld van een multivalued dependency, waarbij de huisdieren en auto's onafhankelijk van elkaar zijn, maar toch in dezelfde tabel staan, wat leidt tot redundantie en verwarring. De oplossing hiervoor is 4NF:

Persoon	Huisdier
Alice	Hond
Alice	Kat
Bob	Hond
Bob	Kat

Persoon	Auto
Alice	BMW
Alice	Audi
Bob	Audi